

0) Problem Forms, Notation, and Core Tools

0.1) Common Mathematical Notation

$$\min_{x \in \mathcal{F}} f(x)$$

subject to ...

decision vector $x = [x_1, \dots, x_n]^T \in \mathbb{R}^n$.

objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

feasible (admissible) set $\mathcal{F}$, i.e. the set of all points satisfying the rules.

0.2) Unconstrained Canonical Form

$$\min_{x \in \mathbb{R}^n} f(x)$$

feasible set $\mathcal{F} = \mathbb{R}^n$, whole space $\mathbb{R}^n$.

- ▶ maximisation can be reduced to minimisation by $\max_x f(x) \Rightarrow \min_x -f(x)$.
- ▶ root-finding can be rewritten as optimisation, e.g. $r(x) = 0 \Rightarrow \min_x \frac{1}{2}r(x)^2$.

0.3) Constrained Canonical Forms

$$\min_x f(x)$$

subject to $g(x) = 0$,

$h(x) \leq 0$,

admissible set $\mathcal{F} = X$, built from the equations and inequalities.

- ▶ here $x \in \mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$, and $h : \mathbb{R}^n \rightarrow \mathbb{R}^p$.
- ▶ the inequality $h(x) \leq 0$ is always understood **componentwise**.

0.3.1) P_0 : full constrained problem

$$\min_x f(x)$$

subject to $g_1(x) = 0, \dots, g_m(x) = 0$,

$h_1(x) \leq 0, \dots, h_p(x) \leq 0$.

0.3.2) P_1 : equality-only form

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g(x) = 0. \end{aligned}$$

0.3.3) $\tilde{P}_1$: equality in inequality-only form

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g(x) \leq 0, \\ & -g(x) \leq 0. \end{aligned}$$

trade-off same variables, but the number of constraints doubles.

0.3.4) P_2 : inequality-only form

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & h(x) \leq 0. \end{aligned}$$

0.3.5) $\tilde{P}_2$: inequality in equality-only form

$$\begin{aligned} \min_{z=[x^T, y^T]^T} \quad & f(x) \\ \text{subject to} \quad & h(x) + Y = 0. \end{aligned}$$

slack variables $Y = [y_1^2, \dots, y_p^2]$.

► each inequality $h_i(x) \leq 0$ can be converted into the equality $h_i(x) + y_i^2 = 0$.

trade-off fewer constraints changes, but more variables are introduced.

Typical modelling translations

square root solve $x^2 - 3 = 0$ by minimising $(x^2 - 3)^2$.

least squares / Fourier fitting pick the coefficients to minimise a sum of squared residuals.

economy dispatch minimise total generation cost subject to power-balance constraints.

support vector machine minimise $\frac{1}{2} \|w\|^2$ subject to margin inequalities.

0.0.2) Generic Optimisation Workflow

Input: initial point x_0 .

1. Compute the information available at x_k .
2. Choose a **search direction** d_k .
3. Choose a **step length** α_k .
4. Update $x_{k+1} = x_k + \alpha_k d_k$.
5. Stop when the relevant stationarity / feasibility test is satisfied.

0.0.3) Types of Minimiser and Saddle Behaviour

global minimiser x^* satisfies $f(x^*) \leq f(y)$ for every feasible y .

strict global min x^* satisfies $f(x^*) < f(y)$ for every feasible $y \neq x^*$.

local minimiser there exists $\rho > 0$ such that $f(x^*) \leq f(y)$ for all feasible $y \in B(x^*, \rho)$.

strict local min there exists $\rho > 0$ such that $f(x^*) < f(y)$ for all feasible $y \neq x^* \in B(x^*, \rho)$.

saddle point a stationary point that function goes down in some directions and up in others.

► **stationary does not imply minimum**, a stationary point may be a min, max, or saddle.

0.3) Mathematical Tools

0.3.1) Derivatives, Quadratic Forms, and Taylor Expansion

row gradient $\frac{\partial f}{\partial x} = \left[\frac{\partial f}{\partial x_1} \quad \cdots \quad \frac{\partial f}{\partial x_n} \right]$.

column gradient $\nabla f(x) = \left(\frac{\partial f}{\partial x} \right)^\top = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$.

Hessian $\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$.

► if $f \in C^2$, then $\nabla^2 f(x) = \nabla^2 f(x)^T$, which makes eigenvalue and definiteness tests possible.

1st-order Taylor's expansion $f(x + s) \approx f(x) + \nabla f(x)^T s$.

2nd-order Taylor's expansion $f(x + s) \approx f(x) + \nabla f(x)^T s + \frac{1}{2} s^T \nabla^2 f(x) s$.

quadratic test function $q(x) = \frac{1}{2} x^T Q x + c^T x + d$, where $Q = Q^T$.

► $\nabla q(x) = Qx + c$ and $\nabla^2 q(x) = Q$, if $Q > 0$, then the **unique minimiser** is $x^* = -Q^{-1}c$.

0.3.2) Test Definiteness

► at a stationary point, the sign of the Hessian is the fastest local classifier.

positive definite all eigenvalues $> 0 \Rightarrow$ local minimiser.

positive semidefinite all eigenvalues $\geq 0 \Rightarrow$ inconclusive, need higher-order test

negative definite all eigenvalues $< 0 \Rightarrow$ local maximiser.

negative semidefinite all eigenvalues $\leq 0 \Rightarrow$ inconclusive, need higher-order test.

positive (negative) indefinite eigenvalues have mixed signs $\Rightarrow$ saddle point.

1×1 case $[a]$ is positive definite if $a > 0$, negative definite if $a < 0$.

2×2 symmetric case $H = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$

► is positive definite if $a > 0$ and $ac - b^2 > 0$;

► is negative definite if $a < 0$ and $ac - b^2 > 0$;

► is indefinite if $ac - b^2 < 0$.

3×3 symmetric case $H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{12} & h_{22} & h_{23} \\ h_{13} & h_{23} & h_{33} \end{bmatrix}$, compute the leading principal minors

$$\Delta_1 = h_{11}, \quad \Delta_2 = \det \begin{bmatrix} h_{11} & h_{12} \\ h_{12} & h_{22} \end{bmatrix}, \quad \Delta_3 = \det(H).$$

► is positive definite if and **only** if $\Delta_1 > 0$, $\Delta_2 > 0$, $\Delta_3 > 0$;

► is negative definite if and **only** if $\Delta_1 < 0$, $\Delta_2 > 0$, $\Delta_3 < 0$;

► verify semidefinite using eigenvalues.

$n \times n$ symmetric case define Δ_k as the determinant of the top-left $k \times k$ block.

- ▶ is positive definite if and **only** if $\Delta_1 > 0, \Delta_2 > 0, \dots, \Delta_n > 0$;
- ▶ is negative definite if and **only** if $\Delta_1 < 0, \Delta_2 > 0, \Delta_3 < 0, \dots$;
- ▶ verify semidefinite using eigenvalues.

saddle test find two directions s_1, s_2 such that $s_1^T H s_1 > 0$ and $s_2^T H s_2 < 0$, then H is indefinite.

singularity if $\det(H) = 0$, second-order test is **inconclusive**, need use higher-order analysis.

0.3.3) Matrix Inversion and Practical Linear Solves

golden rule in algorithms, prefer $Hd = -g$ over $d = -H^{-1}g$.

2×2 inverse

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}, \quad ad - bc \neq 0.$$

3×3 inverse

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

- ▶ compute **determinant** and the **cofactor matrix**

$$C = \begin{bmatrix} + \begin{vmatrix} e & f \\ h & i \end{vmatrix} & - \begin{vmatrix} d & f \\ g & i \end{vmatrix} & + \begin{vmatrix} d & e \\ g & h \end{vmatrix} \\ - \begin{vmatrix} b & c \\ h & i \end{vmatrix} & + \begin{vmatrix} a & c \\ g & i \end{vmatrix} & - \begin{vmatrix} a & b \\ g & h \end{vmatrix} \\ + \begin{vmatrix} b & c \\ e & f \end{vmatrix} & - \begin{vmatrix} a & c \\ d & f \end{vmatrix} & + \begin{vmatrix} a & b \\ d & e \end{vmatrix} \end{bmatrix}.$$

- ▶ then,

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A) = \frac{1}{\det(A)} C^T.$$

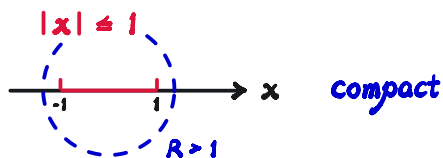
1) Unconstrained Optimisation

1A) Geometric Definitions and Test Objects

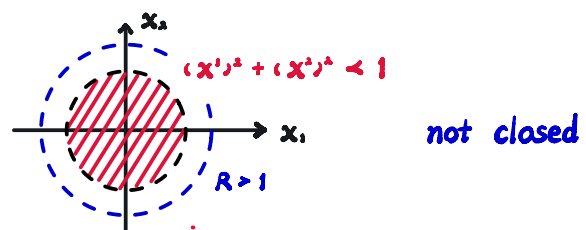
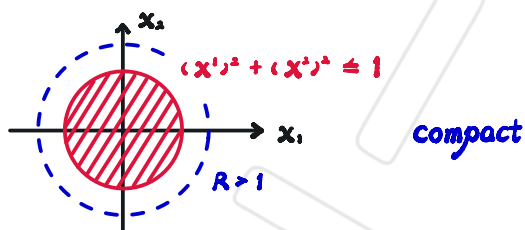
1A.1) Compact Set

- ▶ a set is **compact** if it is **closed** and **bounded**.
- closed** the boundary points belong to the set.
- bounded** the whole set fits inside some ball $\|x\| < R$.
- ▶ if f is **continuous** on a compact set $\mathcal{F}$, then there exist **global minimum** of f in $\mathcal{F}$.
- ▶ search is trapped in a compact set $\Rightarrow$ existence of **minimisers** and **accumulation points**.

$$f: \mathbb{R} \rightarrow \mathbb{R}$$



$$f: \mathbb{R}^2 \rightarrow \mathbb{R}$$



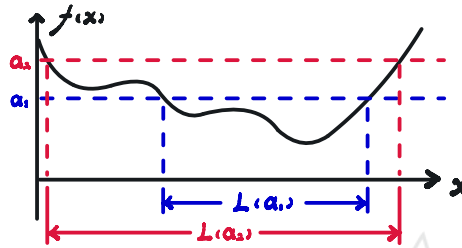
1A.2) Level Set

$$L(\alpha) = \{x \in \mathbb{R}^n : f(x) \leq \alpha\}, \quad \alpha \in \mathbb{R}.$$

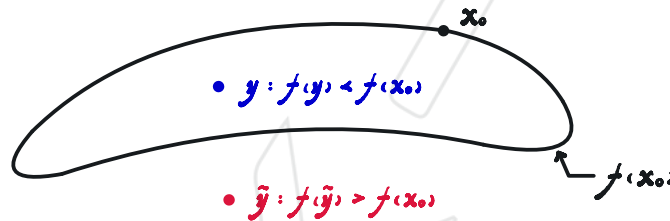
- ▶ the set of all points whose cost is below a chosen threshold.
- ▶ for an algorithm started at x_0 , the important set is $L_0 = L(f(x_0))$.
- ▶ if the method is descent-type, then $x_k \in L_0$ for every k .

nested property

if $\alpha_1 < \alpha_2$,
 then $L(\alpha_1) \subseteq L(\alpha_2)$.

**1A.3) Compact Level Set**

- ▶ a **level set** that is both **closed** and **bounded**.
- ▶ if f is **continuous** and one relevant level set is **compact**, then f has a **global minimiser** on L_0 .
- ▶ this is the standard existence trigger for unconstrained optimisation.

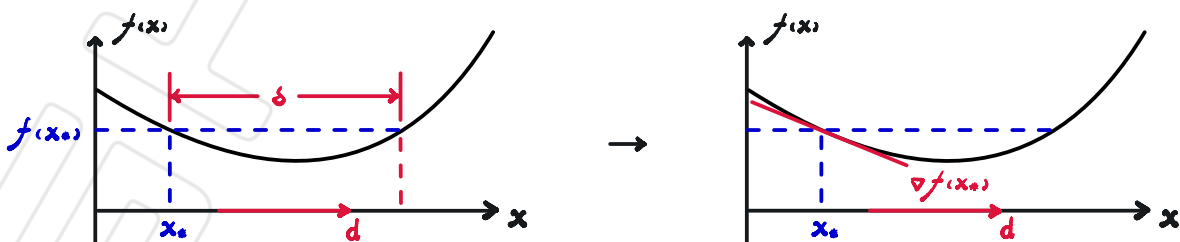
**1A.4) Radial Unboundedness**

$$\|x_k\| \rightarrow \infty \Rightarrow f(x_k) \rightarrow \infty.$$

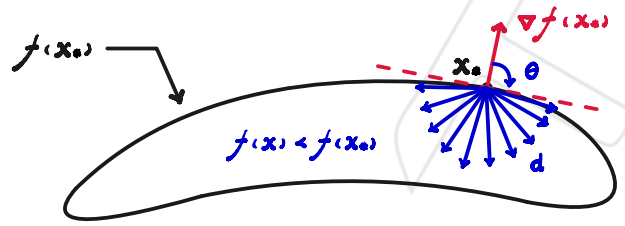
- ▶ for **continuous** f , this is equivalent to saying that **all level sets are compact**.
- ▶ radial unboundedness is a **quick existence test** for global minima in unconstrained problems.

1A.5) Descent Direction

- ▶ d is a **descent direction** if there exists $\delta > 0$ such that $f(x + \lambda d) < f(x)$ for all $\lambda \in (0, \delta)$.



$\nabla f(x)^\top d = \|\nabla f(x)\| \|d\| \cos \theta$,
hence descent means $\cos \theta < 0$



first-order test if $f \in C^1$, then

$$\phi(\alpha) = f(x + \alpha d) - f(x), \quad \phi'(0) = \lim_{\alpha \rightarrow 0^+} \frac{f(x + \alpha d) - f(x)}{\alpha} = \nabla f(x)^\top d.$$

► so $\nabla f(x)^\top d < 0$ immediately proves that d is a descent direction.

second-order rescue if $f \in C^2$ and $\nabla f(x)^\top d = 0$, then inspect

$$\phi''(0) = d^\top \nabla^2 f(x) d.$$

► if $d^\top \nabla^2 f(x) d < 0$, then d is a descent direction (in fact $-d$ is also a descent direction).

► if $d^\top \nabla^2 f(x) d > 0$, then d is not a descent direction.

► if $d^\top \nabla^2 f(x) d = 0$, go to the explicit restriction $\phi(\alpha)$, or perform higher-order test.

restriction trick freeze the point and the direction, then study the one-variable function

$$\phi(\alpha) = f(x + \alpha d) - f(x), \quad \alpha \geq 0.$$

► then d is a descent direction if $\phi(\alpha) < 0$ for all sufficiently small $\alpha > 0$.

gradient direction $d = -\nabla f(x)$ is descent whenever $\nabla f(x) \neq 0$.

Gauss-Newton direction for $f(x) = \frac{1}{2} r(x)^\top r(x)$, let $J(x) = \frac{\partial r(x)}{\partial x}$, then $\nabla f(x) = J(x)^\top r(x)$.

► if $J(x)$ has full column rank, then $J(x)^\top J(x)$ is positive definite, and the direction is

$$d_{GN} = -(J(x)^\top J(x))^{-1} J(x)^\top r(x) \approx -(\nabla^2 f(x))^{-1} \nabla f(x).$$

► since $J(x)^\top J(x)$ is positive definite, its inverse is also positive definite. Hence, if $\nabla f(x) \neq 0$,

$$\nabla f(x)^\top d_{GN} < 0.$$

► therefore, d_{GN} is a descent direction whenever $J(x)$ has full column rank and $J(x)^\top r(x) \neq 0$.

modified Newton any direction of the form $d = -H \nabla f(x)$ with $H > 0$ is descent.

$$\nabla f(x)^\top d = -\nabla f(x)^\top H \nabla f(x) < 0.$$

questions Exercise 2, Exercise 8, Exercise 13, Exercise 16, Exercise 18, Exercise 21.

1B) Existence and Local Optimality Tests

1B.1) First-Order Necessary Condition

$$\nabla f(x^*) = 0.$$

- ▶ if x^* is an **unconstrained local minimiser** and $f \in C^1$, then the gradient must vanish.
- ▶ if $\nabla f(x^*) \neq 0$, then $d = -\nabla f(x^*)$ is a descent direction, which **contradicts** local optimality.
- ▶ the **stationary set** is $\Omega = \{x \in \mathbb{R}^n : \nabla f(x) = 0\}$, contains local min, max, and saddles.

1B.2) Second-Order Necessary Condition

$$\nabla f(x^*) = 0, \quad s^T \nabla^2 f(x^*) s \geq 0 \quad \forall s \in \mathbb{R}^n.$$

- ▶ at a local minimiser the Hessian must be **positive semidefinite**.
- ▶ this result comes from the **second-order Taylor expansion**.

1B.3) Second-Order Sufficient Condition

$$\nabla f(x^*) = 0, \quad s^T \nabla^2 f(x^*) s > 0 \quad \forall s \neq 0.$$

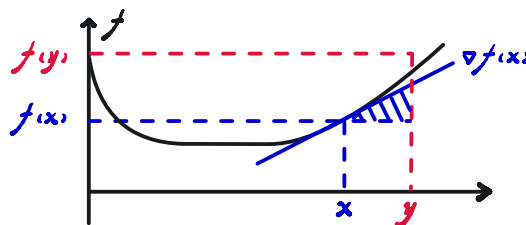
- ▶ this means $\nabla^2 f(x^*) > 0$, x^* is a **strict local minimiser**.

1B.4) Convexity and Strict Convexity

- ▶ f is **convex** if $f(y) - f(x) \geq \nabla f(x)^T (y - x)$ for all x, y .

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

every stationary point
is a global minimiser

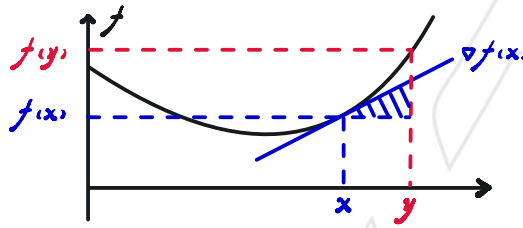


equivalent midpoint form $f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$ for $\theta \in [0, 1]$.

- ▶ f is **strictly convex** if $f(y) - f(x) > \nabla f(x)^T (y - x)$ for all $x \neq y$ and $\theta \in (0, 1)$.

$$f(y) > f(x) + \nabla f(x)^\top (y - x)$$

the minimiser is unique



equivalent midpoint form $f(\theta x + (1 - \theta)y) < \theta f(x) + (1 - \theta)f(y)$ for $\theta \in (0, 1)$.

Hessian test $f \in C^2$ is convex if $\nabla^2 f(x) \geq 0$, strictly convex if $\nabla^2 f(x) > 0$.

quadratic test $\frac{1}{2}x^\top Qx + c^\top x + d$ is convex if $Q \geq 0$, strictly convex if $Q > 0$.

1C) Common Structure of Descent Algorithms

1C.1) General Unconstrained Minimisation Algorithm

$$x_{k+1} = x_k + \alpha_k d_k.$$

- ▶ d_k = search (research) direction.
- ▶ α_k = step size (length) found by a line-search or trust-rule.
- ▶ almost every method in this chapter is nothing more than a different choice of d_k and α_k .
- ▶ if d_k is descent and α_k is chosen properly, then $f(x_{k+1}) < f(x_k)$.

1C.2) Limit Points

- ▶ a point $\bar{x}$ is a **limit (accumulation) point** of $\{x_k\}$ if some subsequence x_{k_j} converges to $\bar{x}$.
- ▶ subsequences may converge to different points even when the full sequence does not settle immediately, e.g. $x_k = (-1)^k$ has **even** accumulation point 1 and **odd** accumulation point -1.
- ▶ that is why many proofs first show **every accumulation point is stationary**.

1C.3) Existence of Accumulation Points

Bolzano-Weierstrass every bounded sequence in $\mathbb{R}^n$ has a convergent subsequence

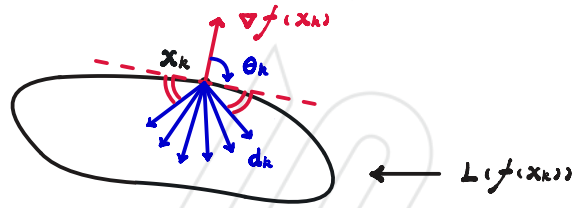
- ▶ if the whole sequence remains inside a **compact level set**, Bolzano-Weierstrass can be used, so at least one convergent subsequence exists.
- ▶ for descent methods, the **monotone decrease** $f(x_{k+1}) \leq f(x_k)$ keeps the iterates inside the compact level set.

1C.4) Condition of Angle

$$\nabla f(x_k)^\top d_k \leq -\varepsilon \|\nabla f(x_k)\| \|d_k\|, \quad \varepsilon > 0.$$

equivalent to $\cos \theta_k \leq -\varepsilon < 0$,

d_k must not become too close to the tangent plane of the level set



► prevents almost-orthogonal directions that create almost no decrease.

1C.5) Global Convergence Theorem for Descent Methods

assumption 1 ∇f is continuous and the level set $L(f(x_0))$ is compact.

assumption 2 the directions satisfy the condition of angle.

assumption 3 the line search gives decrease, so $f(x_{k+1}) < f(x_k)$ whenever $x_k \notin \Omega$.

assumption 4 the step-selection rule is rich enough that $\lim_{k \rightarrow \infty} \frac{\nabla f(x_k)^\top d_k}{\|d_k\|} = 0$.

► accumulation points are stationary, i.e. the algorithm is globally convergent to the stationary set, not automatically to the global minimum.

1C.6) Speed of Convergence

error $e_k = \|x_k - x^*\|$.

► the sequence has order p if $\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k^p} = C_p > 0$.

linear $p = 1, 0 < C_1 < 1$; e.g. $x_k = \frac{1}{10^k} \rightarrow C_1 = \frac{1}{10}$.

sublinear $p = 1, C_1 = 1$; worst, e.g. $x_k = \frac{1}{k} \rightarrow C_1 = 1$.

superlinear $p = 1, C_1 = 0$; ideal, e.g. $x_k = \frac{1}{k!} \rightarrow C_1 = 0$.

divergence $p = 1, C_1 > 1$.

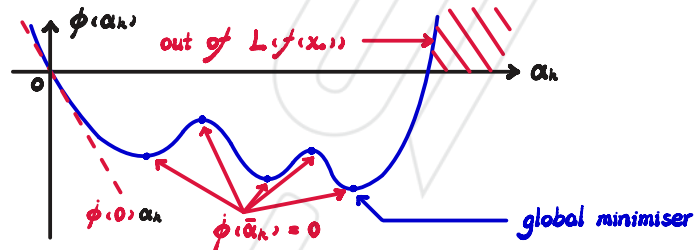
quadratic $p = 2, C_2 > 0$; super fast, e.g. $x_k = \left(\frac{1}{10}\right)^{2^k} \rightarrow C_2 = \frac{e_{k+1}}{e_k^2} = 1$.

- quadratic functions are the clean benchmark for speed of convergence because many methods have exact or near-exact behaviour there.

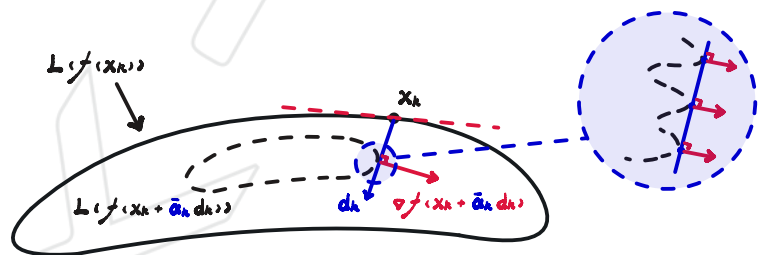
1D) Choosing the Step Length

$$\phi(\alpha) = f(x_k + \alpha d_k) - f(x_k), \quad \alpha \geq 0.$$

one-dimensional reduction



if d_k is descent,
then $\phi'(0) = \nabla f(x_k)^T d_k < 0$



- **exact line search** finds the true minimiser of ϕ .
- **inexact line search** accepts a cheaper step that still gives enough decrease.

1D.1) Exact Line Search

$$\alpha_k \in \arg \min_{\alpha \geq 0} \phi(\alpha) = \arg \min_{\alpha \geq 0} f(x_k + \alpha d_k).$$

complexity one **scalar minimisation** per iteration, often expensive.

conditions (i) $f \in C^1$; (ii) d_k descent direction; (iii) minimiser of ϕ exists.

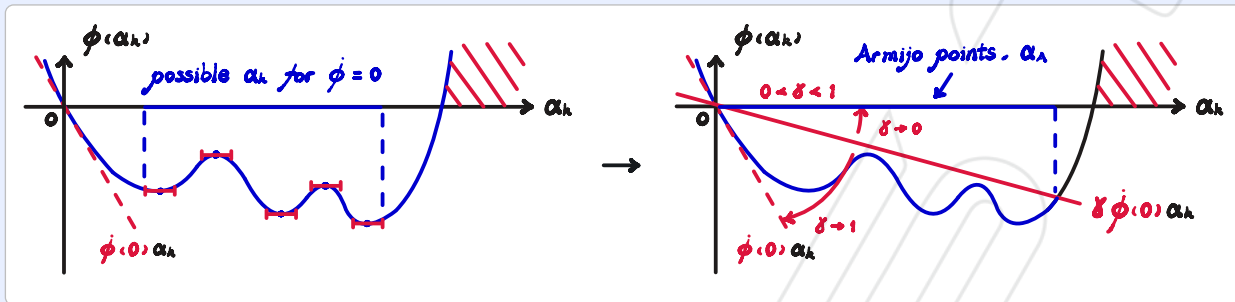
information required $\phi(\alpha) = f(x_k + \alpha d_k) - f(x_k)$, optionally $\phi'(\alpha) = \nabla f(x_k + \alpha d_k)^T d_k$.

best use theory, small problems, quadratic models, and when 1D minimisation is cheap.

questions Exercise 1, Exercise 3, Exercise 4, Exercise 16, Exercise 22, Exercise 23; 2015 Q2, 2016 Q1, 2018 Q1, 2023 Q4.

1D.2) Armijo method

► pick a large trial $\alpha_k > 0 \rightarrow$ check the Armijo inequality $\rightarrow$ if true, stop.



$$f(x_k + \alpha_k d_k) - f(x_k) \leq \gamma \alpha_k \nabla f(x_k)^\top d_k$$

► if false $\rightarrow \alpha_k \leftarrow \sigma \alpha_k \rightarrow$ test again.



complexity cheap backtracking, a few function evaluations per iteration.

conditions (i) $f \in C^1$; (ii) compact level set $L(f(x_0))$; (iii) d_k descent direction; (iv) choose $a > 0, \sigma \in (0, 1), \gamma \in (0, \frac{1}{2})$; (v) $C_2 \|\nabla f(x_k)\|^q \leq \|d_k\| \leq C_1$.

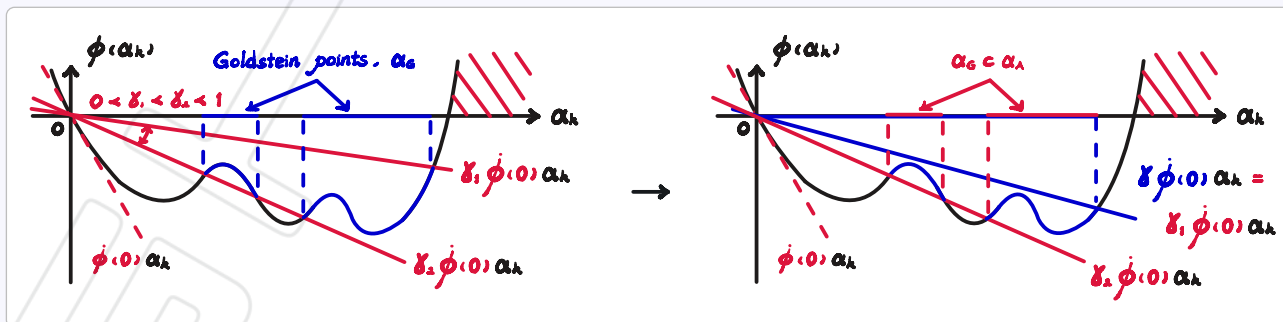
information required $f(x_k + \alpha d_k), f(x_k), \nabla f(x_k)^\top d_k$.

best use default safe line search when you want **robustness** more than perfect step accuracy.

1D.3) Goldstein condition

► choose a trial $\alpha_k \rightarrow$ accept only if α_k lies inside the Goldstein region.

$$\alpha_k \gamma_2 \nabla f(x_k)^\top d_k \leq f(x_k + \alpha_k d_k) - f(x_k) \leq \alpha_k \gamma_1 \nabla f(x_k)^\top d_k, \quad 0 < \gamma_1 < \gamma_2 < 1.$$



complexity often fewer trial points than simple Armijo backtracking.

conditions (i) $f \in C^1$; (ii) compact level set $L(f(x_0))$; (iii) d_k descent direction; (iv) choose $0 < \gamma_1 < \gamma_2 < 1$.

information required $f(x_k + \alpha d_k)$, $f(x_k)$, $\nabla f(x_k)^\top d_k$.

best use when you want a line search that avoids tiny accepted steps more explicitly.

1D.4) Wolfe condition (Extension)

► Armijo and Goldstein control sufficient decrease, Wolfe adds a curvature condition, so the accepted step is also compatible with the next search direction.

$$f(x_k + \alpha_k d_k) - f(x_k) \leq c_1 \alpha_k \nabla f(x_k)^\top d_k,$$

$$\nabla f(x_k + \alpha_k d_k)^\top d_k \geq c_2 \nabla f(x_k)^\top d_k,$$

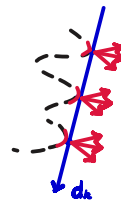
$$0 < c_1 < c_2 < 1.$$

► the second inequality keeps the new slope from being too negative.

► strong Wolfe replaces the curvature test by

$$|\nabla f(x_k + \alpha_k d_k)^\top d_k| \leq c_2 |\nabla f(x_k)^\top d_k|.$$

this version keeps the new gradient almost orthogonal to the search direction and is natural for Polak-Ribière and quasi-Newton methods.



1.1) The Gradient Method

1.1) The Gradient Method

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = -\nabla f(x_k).$$

complexity $O(n)$ once $\nabla f(x_k)$ is available.

conditions (i) $f \in C^1$; (ii) compact level set $L(f(x_0))$.

α_k any line search rule with $f(x_k + \alpha_k d_k) \leq f(x_k)$, exact line search is common in theory.

information required at each step $f(x_k), \nabla f(x_k)$.

computation to find the research direction $d_k = -\nabla f(x_k)$.

quadratic convergence quadratic case: **linear**; slow if $\kappa \gg 1$; one step if $\kappa = 1$.

Derivation sketch for the quadratic test function

► for $f(x) = \frac{1}{2}x^T Q x + c^T x + d$ with $Q = Q^T > 0$

► choose $d_k = -\nabla f(x_k)$ and minimise $\phi(\alpha) = f(x_k - \alpha \nabla f(x_k)) - f(x_k)$.

$$\phi'(\alpha) = -\nabla f(x_k)^T \nabla f(x_k) + \alpha \nabla f(x_k)^T Q \nabla f(x_k).$$

► set $\phi'(\alpha_k) = 0$ to obtain the exact line-search step

$$\alpha_k = \frac{\nabla f(x_k)^T \nabla f(x_k)}{\nabla f(x_k)^T Q \nabla f(x_k)}.$$

Condition-number effect

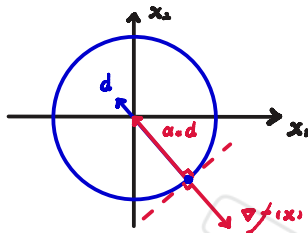
► the ratio $\kappa = \lambda_M / \lambda_m$ measures how stretched the level ellipsoids are.

$$\frac{e_{k+1}}{e_k} \leq \sqrt{\kappa} \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}$$

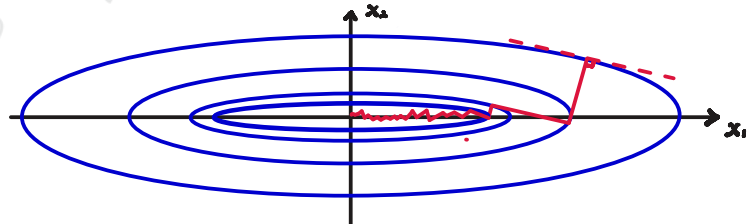
► if $\kappa = 1$, the level sets are spherical and exact line search reaches the minimiser in **one step**.

► if $\kappa \gg 1$, the method shows the familiar **zig-zag** behaviour and convergence becomes slow.

$\kappa = 1$



$\kappa \gg 1$



► the gradient method is safe but often inefficient on ill-conditioned quadratic models.

general convergence every accumulation point is stationary; usually slow on ill-conditioned level sets.

suitability best for large problems with cheap gradients, or as a safe fallback direction.

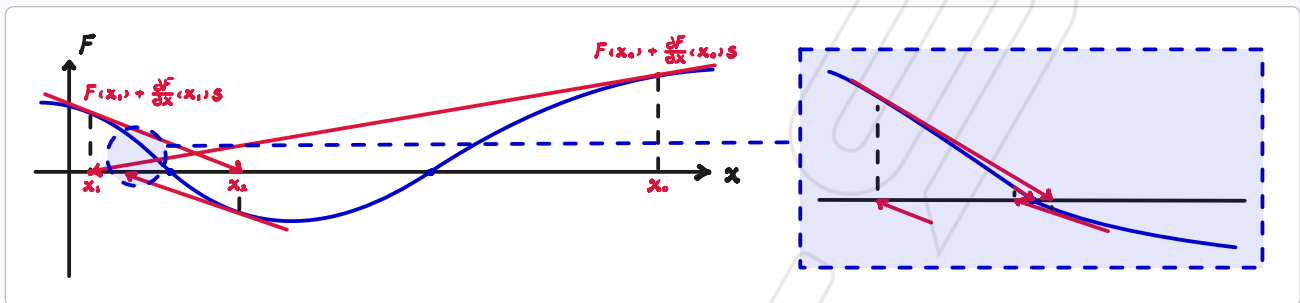
questions Exercise 1, Exercise 4, Exercise 11, Exercise 12, Exercise 22, Exercise 23; 2015 Q2, 2016 Q1, 2017 Q1, 2018 Q1, 2023 Q4.

1.2) Newton-Type Methods

1.2.1) Newton-Raphson iteration

Root-finding form first

$$F(x) = 0 \quad \Rightarrow \quad x_{k+1} = x_k - \left[\frac{\partial F}{\partial x}(x_k) \right]^{-1} F(x_k).$$



- ▶ for a scalar equation $F(x) = 0$, this becomes $x_{k+1} = x_k - \frac{F(x_k)}{F'(x_k)}$.
- ▶ optimisation is obtained by choosing stationarity equation $F(x) = \nabla f(x)$.

Derivation sketch

$$F(x_k + s) = F(x_k) + \frac{\partial F}{\partial x}(x_k)s + \text{h.o.t.}$$

- ▶ ignore the higher-order terms and force the linear model to be zero.

$$F(x_k) + \frac{\partial F}{\partial x}(x_k)s_k = 0 \quad \Rightarrow \quad s_k = - \left[\frac{\partial F}{\partial x}(x_k) \right]^{-1} F(x_k).$$

Quadratic-model derivation behind Newton's step

$$f(x_k + s) \approx f(x_k) + \nabla f(x_k)^T s + \frac{1}{2} s^T \nabla^2 f(x_k) s.$$

- ▶ set the gradient of this quadratic model with respect to s equal to zero.

$$\nabla f(x_k) + \nabla^2 f(x_k)s = 0 \quad \Rightarrow \quad \nabla^2 f(x_k)s = -\nabla f(x_k).$$

- ▶ so Newton's step is exactly the **minimiser of the local quadratic approximation**.

1.2.2) Newton's method

$$x_{k+1} = x_k + s_k, \quad s_k = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

complexity dense Hessian solve is typically $O(n^3)$, storage $O(n^2)$.

conditions (i) $f \in C^2$; (ii) $\nabla^2 f(x_k)$ invertible where Newton step is used; (iii) $\nabla^2 f(x_k) > 0$ for local-minimiser interpretation; (iv) local quadratic rate: $\nabla f(x^*) = 0$, $\nabla^2 f(x^*)$ nonsingular, Hessian locally Lipschitz; (v) x_0 close to x^* .

α_k pure Newton uses $\alpha_k = 1$, globalised Newton uses a line search.

information required at each step $f(x_k)$, $\nabla f(x_k)$, $\nabla^2 f(x_k)$.

computation to find the research direction $\nabla^2 f(x_k) d_k = -\nabla f(x_k)$.

quadratic convergence local quadratic; one step on quadratic $Q > 0$.

general convergence no global guarantee.

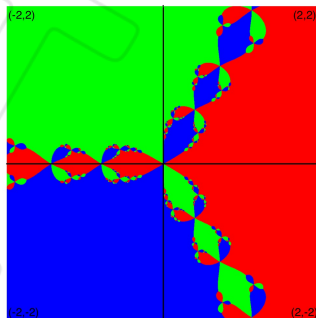
suitability small and medium scale problems where $f \in C^2$ and high local speed matters.

questions Exercise 2, Exercise 7, Exercise 15, Exercise 18, Exercise 26; 2019 Q1, 2020 Q4, 2021 Q4, 2022 Q1, 2015 Q1.

1.2.3) Newton's Fractal (Extension)

► in root-finding with several roots, the complex plane splits into coloured attraction regions: different initial points converge to different roots.

for $p(x) = x^3 - 1$



red → converge to root $x = 1$

blue → converge to root $x = -\frac{1}{2} - \frac{\sqrt{3}}{2}i$

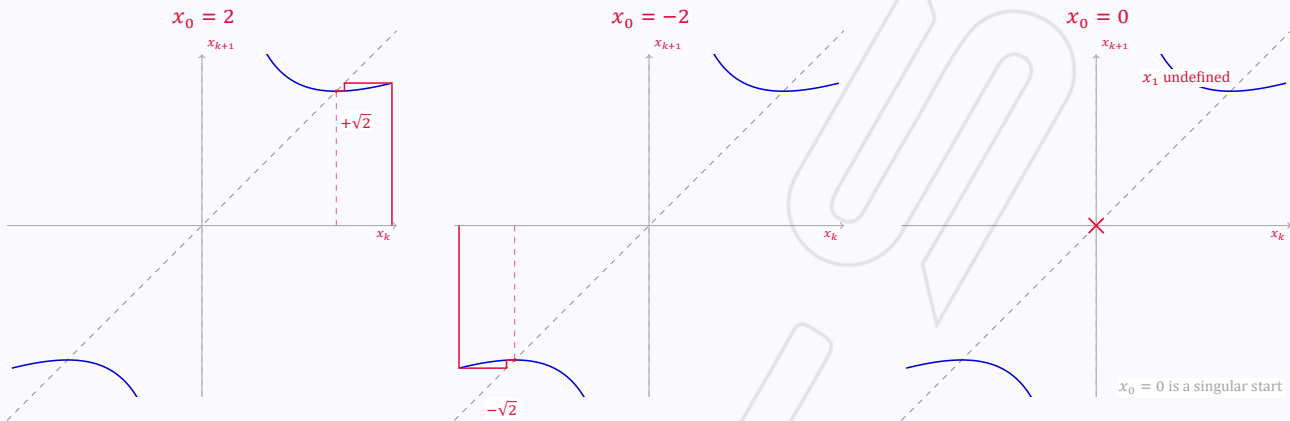
green → converge to root $x = -\frac{1}{2} + \frac{\sqrt{3}}{2}i$

► pure Newton can be very sensitive to the initial point.

Spider diagrams

e.g. to find $\sqrt{2}$, apply Newton-Raphson to $F(x) = x^2 - 2 = 0$:

$$x_{k+1} = x_k - \left[\frac{\partial F}{\partial x}(x_k) \right]^{-1} F(x_k) = \frac{1}{2} \left(x_k + \frac{2}{x_k} \right), \quad x^* = \pm\sqrt{2}.$$



► the three starts behave differently:

(i) $2 \mapsto 1.5 \mapsto 1.4167 \mapsto 1.4142 \rightarrow +\sqrt{2}$,

(ii) $-2 \mapsto -1.5 \mapsto -1.4167 \mapsto -1.4142 \rightarrow -\sqrt{2}$,

(iii) $x_0 = 0$ fails immediately because the Newton update contains $2/x_k$.

► the initial condition decides which root is approached, or iteration not even defined.

1.2.4) Newton-gradient algorithm

► check $\nabla^2 f(x_k)$:

singular $\rightarrow d_k = -\nabla f(x_k)$.

non-singular $\rightarrow s_k = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$.

► then test the research direction:

$\nabla f(x_k)^\top s_k < 0 \rightarrow d_k = s_k$.

$\nabla f(x_k)^\top s_k \geq 0 \rightarrow d_k = -\nabla f(x_k)$.

► finally $\rightarrow$ line search along $d_k \rightarrow x_{k+1} = x_k + \alpha_k d_k$.

► try Newton first, but keep a gradient fallback whenever the Newton solve is singular or almost orthogonal to the gradient.

complexity same dense cost class as Newton, $O(n^3)$, plus line search.

conditions (i) $f \in C^2$; (ii) Newton local hypotheses for the final quadratic phase.

α_k line search, usually started with initial guess $\alpha = 1$, Armijo is a common safe choice.

information required at each step $f(x_k), \nabla f(x_k), \nabla^2 f(x_k)$.

computation to find the research direction $g_k = \nabla f(x_k), H_k = \nabla^2 f(x_k)$.

quadratic convergence recovers Newton's local quadratic rate once $\alpha_k = 1$ is accepted.

general convergence global stationary-point convergence via safeguard + line search.

suitability practical **robust version of Newton** when you want quadratic speed of convergence near the solution but safety away from it.

questions 2015 Q1 (modified Newton), 2022 Q1 (Newton + alternative).

1.2.5) Trust region

► do not trust the quadratic Taylor model everywhere, trust it only in a ball around x_k .

trust region $\{s : \|s\| \leq a_k\}$, inside this ball the quadratic model is regarded as reliable.

► build the quadratic model $q_k(s) \rightarrow \text{solve } \min_{\|s\| \leq a_k} q_k(s) \rightarrow \text{obtain } s_k$.

$$\rho_k = \frac{f(x_k + s_k) - f(x_k)}{q_k(s_k) - q_k(0)} = \frac{\text{actual reduction}}{\text{expected reduction}}$$

where

$$q_k(s) = f(x_k) + \nabla f(x_k)^T s + \frac{1}{2} s^T \nabla^2 f(x_k) s, \quad q_k(0) = f(x_k)$$

► then update by cases

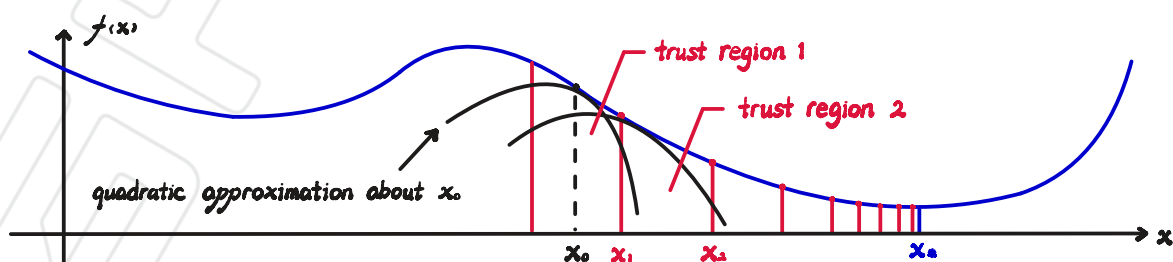
$$\rho_k \leq 0 \Rightarrow a_{k+1} = \|s_k\|/4, \quad x_{k+1} = x_k,$$

$$0 < \rho_k < 1/4 \Rightarrow a_{k+1} = \|s_k\|/4, \quad x_{k+1} = x_k + s_k,$$

$$1/4 < \rho_k < 3/4 \Rightarrow a_{k+1} = a_k, \quad x_{k+1} = x_k + s_k,$$

$$\rho_k > 3/4 \text{ and } \|s_k\| = a_k \Rightarrow a_{k+1} = 2a_k, \quad x_{k+1} = x_k + s_k.$$

in practice, one can choose different value than 1/4 and 3/4.



complexity one quadratic subproblem per iteration, dense exact solve is usually $O(n^3)$.

conditions (i) $f \in C^2$; (ii) $a_0 > 0, a_k > 0$; (iii) trust-region subproblem solved exactly or accurately enough; (iv) compact level set + continuous Hessian; (v) local minimiser x^* with $\nabla^2 f(x^*) \succ 0$.

α_k absorbed into s_k , there is no separate line search in the basic form.

information required at each step $f(x_k), \nabla f(x_k), \nabla^2 f(x_k), a_k$.

computation to find the research direction build $q(s)$ at x_k , solve $\min_{\|s\| \leq a_k} q(s)$, and call the solution s_k .

quadratic convergence local quadratic once the trust-region step becomes the full Newton step; if f is quadratic, then $\rho_k = 1$.

general convergence robust Newton globalisation; accumulation points are stationary.

suitability medium-scale problems where you want Newton-like local speed but safer behaviour away from the solution, it may shorten the Newton step even when the full Newton step could also work.

1.3) Conjugacy-Based Methods

1.3.1) Q-conjugacy

$$d_i^\top Q d_j = 0 \quad (i \neq j)$$

- ▶ if $Q = I$, this reduces to ordinary orthogonality.
- ▶ if $Q = Q^\top \succ 0$, mutually Q -conjugate nonzero directions are linearly independent.
- ▶ in $\mathbb{R}^n$, we have most n Q -conjugate vectors.

example if $c_1 d_1 + c_2 d_2 = 0$, multiply by $d_1^\top Q$ to get $c_1 d_1^\top Q d_1 = 0$, so $c_1 = 0$; then $c_2 = 0$ follows as well (the same idea works for more directions).

1.3.2) Conjugate directions method

$$x_{k+1} = x_k + \alpha_k d_k, \quad \alpha_k = -\frac{\nabla f(x_k)^\top d_k}{d_k^\top Q d_k}$$

- ▶ with prescribed conjugate directions $d_0, \dots, d_{n-1}$.

complexity for dense quadratic problems, $O(n^2)$ work per step once the directions are known.

conditions (i) quadratic objective $f(x) = \frac{1}{2}x^T Qx + c^T x + d$; (ii) $Q = Q^T > 0$; (iii) nonzero mutually Q -conjugate directions.

α_k exact line search along d_k , which has the closed form above.

information required at each step $Q, \nabla f(x_k)$.

computation to find the research direction $d_0, \dots, d_{n-1}, \quad d_i^T Q d_j = 0 \ (i \neq j)$.

quadratic convergence at most n steps.

suitability mainly theoretical foundation and a guide for building conjugate gradient methods.

questions 2023 Q4 (explicit conjugate-direction computation).

1.3.3) Conjugate gradient method

$$d_0 = -\nabla f(x_0), \quad x_{k+1} = x_k + \alpha_k d_k, \quad d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k.$$

► d_k start from the negative gradient, then becomes Q -conjugate to the previous ones.

complexity dense quadratic case $O(n^2)$ per step, or $O(\text{nnz}(Q))$ in sparse form, low memory.

conditions (i) quadratic objective $f(x) = \frac{1}{2}x^T Qx + c^T x + d$; (ii) $Q = Q^T > 0$; (iii) $d_0 = -\nabla f(x_0)$.

α_k exact line search

$$\alpha_k = -\frac{\nabla f(x_k)^T d_k}{d_k^T Q d_k}.$$

► for the quadratic version, the recurrence coefficient is

$$\beta_k = \frac{\nabla f(x_{k+1})^T Q d_k}{d_k^T Q d_k}.$$

information required at each step $Q, \nabla f(x_k), \nabla f(x_{k+1}), d_k$.

computation to find the research direction $\beta_k, d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k$.

quadratic convergence at most n steps.

suitability large quadratic problems, e.g. sparse SPD systems, where Newton is too expensive.

questions 2023 Q4.

Derivation sketch

► for the quadratic test function, exact line search along d_k gives

$$\alpha_k = -\frac{\nabla f(x_k)^\top d_k}{d_k^\top Q d_k}.$$

► since $\nabla f(x) = Qx + c$,

$$\nabla f(x_{k+1}) = \nabla f(x_k) + \alpha_k Q d_k.$$

► use this identity together with the conditions $d_k^\top \nabla f(x_{k+1}) = 0$ and $d_k^\top Q d_{k+1} = 0$ to obtain

$$\beta_k = \frac{\nabla f(x_{k+1})^\top Q d_k}{d_k^\top Q d_k}.$$

► this is the algebraic reason why the new direction stays Q -conjugate to the previous ones.

1.3.4) Polak-Ribiere algorithm

$$d_0 = -\nabla f(x_0), \quad d_k = -\nabla f(x_k) + \beta_{k-1} d_{k-1}, \quad \beta_{k-1} = \frac{\nabla f(x_k)^\top (\nabla f(x_k) - \nabla f(x_{k-1}))}{\|\nabla f(x_{k-1})\|^2}.$$

► d_k update as negative gradient plus a memory term from the previous direction.

complexity $O(n)$ vector to update the direction once the gradient is known, plus line search.

conditions (i) $f \in C^1$; (ii) restart / angle safeguard if global convergence is needed.

α_k exact for quadratic, sufficiently accurate (Wolfe) in practice for nonquadratic problems.

information required at each step $f(x_k)$, $\nabla f(x_k)$, $\nabla f(x_{k-1})$, d_{k-1} .

computation to find the research direction β_{k-1}, d_k .

quadratic convergence finite-step behaviour can be recovered with suitable restart structure.

general convergence usually faster than gradient method; global convergence needs restart / angle safeguard.

suitability medium and large smooth problems when Hessians are unavailable and you want something lighter than BFGS.

1.4) Variable-Metric (Quasi-Newton) Methods

1.4.1) Quasi-Newton equation

for $f(x) = \frac{1}{2}x^T Qx + c^T x + d$ with $Q = Q^T > 0$,

$$\nabla f(x_{k+1}) - \nabla f(x_k) = (Qx_{k+1} + c) - (Qx_k + c) = Q(x_{k+1} - x_k)$$

therefore,

$$H_{k+1}\gamma_k = \delta_k, \quad \gamma_k = \nabla f(x_{k+1}) - \nabla f(x_k), \quad \delta_k = x_{k+1} - x_k.$$

► make H_k behave like $[\nabla^2 f(x_k)]^{-1}$ without computing the true Hessian.

► quasi-Newton methods fit the same generic update

$$x_{k+1} = x_k - \alpha_k H_k \nabla f(x_k).$$

► the secant (quasi-Newton) equation does **not** determine H_{k+1} uniquely, so **DFP** and **BFGS** are two particular rank-two update choices.

Emphasis

► if $H_k > 0$ and the line search satisfies Wolfe, then $d_k = -H_k \nabla f(x_k)$ is a descent direction.

1.4.2) Davidon-Fletcher-Powell (DFP) method

$$H_{k+1} = H_k + \frac{\delta_k \delta_k^T}{\delta_k^T \gamma_k} - \frac{H_k \gamma_k \gamma_k^T H_k}{\gamma_k^T H_k \gamma_k}.$$

complexity $O(n^2)$ storage and update per iteration, plus line search.

conditions (i) $f \in C^1$; (ii) $H_0 = H_0^T > 0$; (iii) curvature condition $\delta_k^T \gamma_k > 0$; (iv) strict convexity for the global result.

α_k line search along d_k , exact line search gives the cleanest quadratic theory.

information required at each step $f(x_k)$, $\nabla f(x_k)$, H_k , δ_k , γ_k .

computation to find the search direction inverse-Hessian approximation.

quadratic convergence quadratic + exact line search: **at most n steps**, $H_n = Q^{-1}$.

general convergence global for strictly convex f with exact line search; local rate **superlinear**.

suitability small and medium scale smooth problems, BFGS is usually preferred.

1.4.3) Broyden-Fletcher-Goldfarb-Shanno (BFGS) method

$$H_{k+1} = (I - \rho_k \delta_k \gamma_k^\top) H_k (I - \rho_k \gamma_k \delta_k^\top) + \rho_k \delta_k \delta_k^\top, \quad \rho_k = \frac{1}{\gamma_k^\top \delta_k}.$$

complexity $O(n^2)$ storage and update per iteration, plus line search.

conditions (i) $f \in C^1$; (ii) $H_0 = H_0^\top > 0$; (iii) curvature condition $\gamma_k^\top \delta_k > 0$; (iv) convexity for the global result.

α_k line search along d_k , sufficiently accurate non-exact search is enough in the theory.

information required at each step $f(x_k)$, $\nabla f(x_k)$, H_k , δ_k , γ_k .

computation to find the research direction update H_k with the BFGS formula written above.

quadratic convergence quadratic case: strong variable-metric behaviour; finite-step result not emphasised here.

general convergence global for convex functions with sufficiently accurate non-exact line search; local rate **superlinear**.

suitability small and medium scale smooth optimisation, **best all-round method** when Hessians are unavailable but gradients are reliable.

1.5) Derivative-Free and Approximate-Derivative Methods

1.5.1) Direct search method

complexity dominated by repeated function evaluations, no fixed clean polynomial formula because the number of trials depends on the search history.

conditions (i) only function values

► convergence theory is strongest when cycles through n linearly independent directions.

d_k trial directions chosen by the direct-search rule, not by gradients.

α_k line search without derivatives or a geometric trial-step rule.

information required at each step $f(x_k + \alpha d)$, $d \in \mathcal{D}_k$.

computation to find the research direction $d_k = \arg \min_{d \in \mathcal{D}_k} f(x_k + \alpha d)$.

quadratic convergence no special quadratic-case theorem.

general convergence stationary-point convergence under suitable direction / step conditions; usually slow.

suitability black-box objectives, noisy settings, or situations where derivatives are unavailable, usually only for few variables.

questions Exercise 5.

1.5.2) Finite difference approximation method

$$\frac{\partial f(x)}{\partial x_i} \approx \frac{f(x + te_i) - f(x)}{t}, \quad t > 0.$$

complexity roughly n extra function evaluations for a forward-difference gradient.

conditions (i) $f \in C^1$, preferably smoother; (ii) $t > 0$ chosen carefully; (iii) outer method assumptions still required.

d_k obtained by plugging the approximate gradient into a derivative-based outer method.

α_k inherited from the outer optimisation method.

information required at each step $f(x_k), f(x_k + te_i), i = 1, \dots, n$.

computation to find the research direction $\tilde{g}_i, d_k = -\tilde{g}$.

quadratic convergence no clean quadratic-case theorem; finite-difference error prevents exact derivative behaviour.

general convergence works on smooth problems, but accuracy is limited by truncation, round-off, and noise.

1.5.3) Coordinate direction method

$$d_k = e_j, \quad j = 1, 2, \dots, n \text{ cyclically.}$$

complexity one sweep uses n one-dimensional searches, cost dominated by evaluate function.

conditions (i) only function values / one-dimensional evaluations; (ii) cyclic directions $e_1, \dots, e_n$.

d_k the current basis vector e_j .

α_k derivative-free line search along e_j .

information required at each step $f(x_k + \alpha e_j)$.

computation to find the research direction $d_k = e_j$, $j = 1, \dots, n$ cyclically.

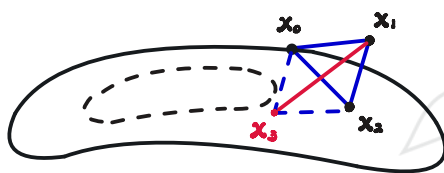
quadratic convergence no special quadratic-case theorem; quadratic problems may zig-zag badly.

general convergence stationary-point convergence under the general theorem; usually slow and axis-dependent.

suitability very small problems or whose coordinates have a strong natural meaning.

questions Exercise 5.

1.5.4) Simplex algorithm



for $n = 2$, pick 3 points, x_0 , x_1 , and x_2 ←
 → $f(x_1) > f(x_0) > f(x_2)$
 → mirroring x_1 about x_0x_2 to get x_3 ←

Warning

► the mirrored point can remain the worst point, so practical fixes are often needed.

complexity f value comparisons on $n + 1$ simplex vertices each move, good only for small n .

conditions (i) only function values; (ii) start with $n + 1$ affinely independent points; (iii) heuristic method, no strong clean general theorem emphasised in the notes.

d_k implicit, obtained by reflecting the worst vertex through the simplex centre.

α_k encoded in the reflection / shrink factors, not as a standard line-search scalar.

quadratic convergence no quadratic-case theorem.

general convergence heuristic only; acceptable mainly for small n and approximate solutions.

suitability few variables, rough optimisation, derivative-free experimentation.

2) Nonlinear Programming

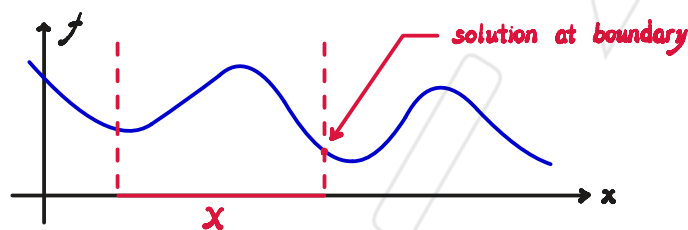
2A) Feasible-Set Geometry, Regularity, and KKT Tests

2A.1) Admissible Set

- ▶ the set of all points satisfying the constraints is the **admissible set** X .
- ▶ a constrained local minimiser x^* satisfies $f(y) \geq f(x^*)$ for all **admissible** $y \in X \cap B(x^*, \rho)$.
- ▶ a constrained global minimiser satisfies $f(y) \geq f(x^*)$ for all $y \in X$.

Reminder

- ▶ the constrained minimiser can lie on the **boundary** of the admissible set.



2A.2) Active Constraint

- ▶ equality constraints are **always active**.
- ▶ the inequality $h_i(x) \leq 0$ is **active** at $\tilde{x}$ if $h_i(\tilde{x}) = 0$.
- ▶ **only active inequalities (equalities) matter** in the first-order and second-order local tests.

set of active constraint $I_a(\tilde{x}) = \{i \in \{1, \dots, p\} : h_i(\tilde{x}) = 0\}$.

vector of active constraint $h_a(\tilde{x}) = \{h_i(\tilde{x}) \mid i \in I_a(\tilde{x})\}$.

2A.3) Regular Point

- ▶ a feasible point $\tilde{x}$ is **regular** if the gradients of all equality constraints and all **active** inequality constraints are **linearly independent** at $\tilde{x}$.
- ▶ if there is **no active inequality** and **no equality constraint**, then the point is regular.

- ▶ find active constraints $\rightarrow$ differentiate $\rightarrow$ build a matrix $\rightarrow$ check independence.

column form put the active gradient vectors as columns:

$$A(\tilde{x}) = \begin{bmatrix} \nabla g_1(\tilde{x}) & \cdots & \nabla g_m(\tilde{x}) & \nabla h_{i_1}(\tilde{x}) & \cdots & \nabla h_{i_r}(\tilde{x}) \end{bmatrix}, \quad I_a(\tilde{x}) = \{i_1, \dots, i_r\}.$$

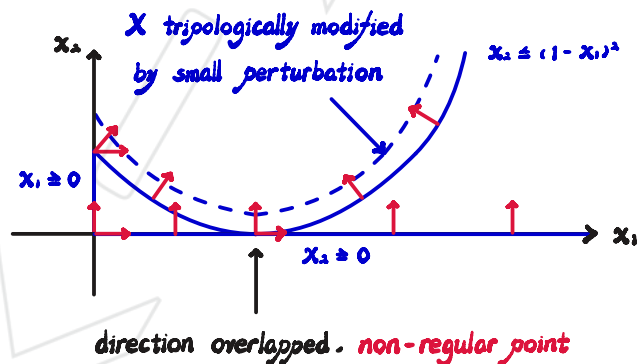
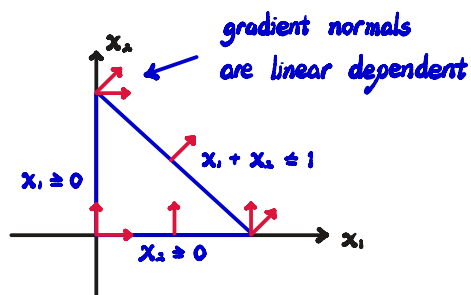
- ▶ then $\tilde{x}$ is regular if $\text{rank } A(\tilde{x}) = m + r$.
- ▶ the $m + r$ active gradients do **not** depend on each other.

row form equivalently, put one active constraint derivative in each row:

$$\begin{bmatrix} \frac{\partial g(\tilde{x})}{\partial x} \\ \frac{\partial h_a(\tilde{x})}{\partial x} \end{bmatrix}.$$

- ▶ now check **row rank** = $m + r$. This is the same test, because row rank = column rank.

- ▶ find active constraints $\rightarrow$ plot them $\rightarrow$ check independence.



Why it matters

- ▶ regularity is the clean assumption used before writing KKT
- ▶ it also gives the usual uniqueness of the multipliers in the standard theorem.

questions Exercise 27, Exercise 30, Exercise 37, Exercise 38.

2A.4) First-Order Necessary Condition (Kuhn-Tucker (KKT) condition)

Lagrangian

$$L(x, \lambda, \rho) = f(x) + \lambda^T g(x) + \rho^T h(x).$$

KKT conditions for P_0 at a regular constrained local minimiser

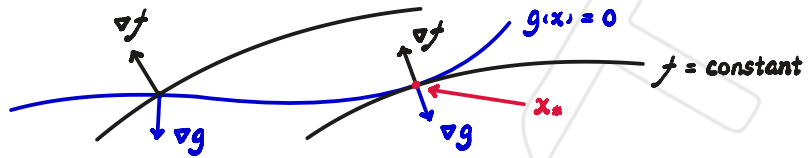
$$\nabla_x L(x^*, \lambda^*, \rho^*) = 0,$$

$$g(x^*) = 0, \quad \text{Lagrangian multiplier } \lambda^* \in \mathbb{R}^m,$$

$$h(x^*) \leq 0, \quad \text{Kuhn-Tucker multiplier } \rho^* \in \mathbb{R}^p, \rho^* \geq 0$$

$$\rho_i^* h_i(x^*) = 0, \quad i = 1, \dots, p.$$

$\nabla_x L = 0$ only possible if
 ∇f is parallel to ∇g



complementarity $\rho_i^* h_i(x^*) = 0$, so each inequality is either inactive ($h_i < 0$) with $\rho_i^* = 0$, or active ($h_i = 0$) with $\rho_i^* \geq 0$.

► if there are no active constraints, KKT collapses to $\nabla f(x^*) = 0$.

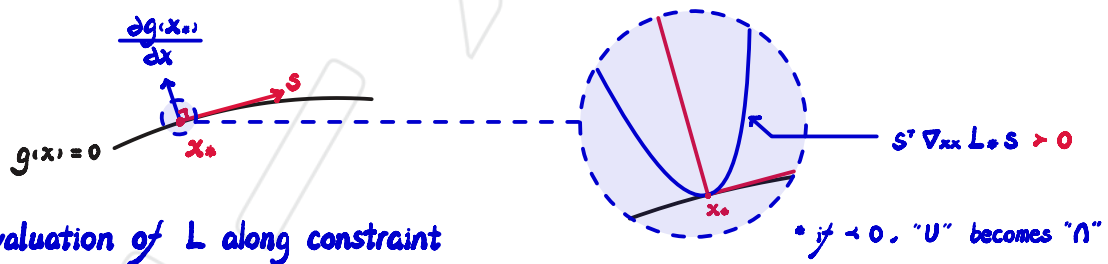
2A.5) Second-Order Sufficient Condition

- first-order necessary condition first.
- need **strict complementarity**: every active inequality satisfies $h_i(x^*) = 0$ and $\rho_i^* > 0$.
- then require

$$s^T \nabla_{xx}^2 L(x^*, \lambda^*, \rho^*) s > 0$$

- for every tangent direction $s \neq 0$ satisfying the linearised active constraints.

$$\begin{bmatrix} \frac{\partial g(x^*)}{\partial x} \\ \frac{\partial h_a(x^*)}{\partial x} \end{bmatrix} s = 0.$$



- x^* is a **strict constrained local minimiser**.

2.1) Unconstrained Reformulation Methods

2.1.1) Sequential Penalty Method

$$F_\varepsilon(x) = f(x) + \frac{1}{\varepsilon} p(x), \quad p(x) = \sum_{i=1}^m g_i(x)^2 + \sum_{i=1}^p \max(0, h_i(x))^2.$$

- $q(x) = 0$ if $x \in X$, and $q(x) = +\infty$ otherwise, but such form is numerically unstable.
- soft penalty replaces q by the real-valued function p above.

Warning

► $\max(0, h_i(x))$ is not differentiable at the boundary, squared or higher-power versions are smoother, but they also become flatter near the boundary, so the resulting penalised problem can become numerically awkward.

complexity $N_{\text{outer}} \times \text{cost}(\text{inner solver})$.

conditions (i) penalty $p(x)$: zero on X , positive outside; (ii) $\varepsilon_k \downarrow 0$; (iii) each penalised sub-problem has a minimiser; (iv) compact near-feasible / sublevel sets; (v) strict constrained local minimiser x^* .

Main weakness

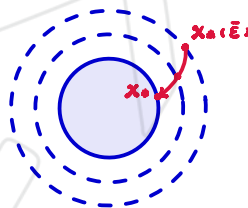
► as $\varepsilon \rightarrow 0$, the penalised problem becomes ill-conditioned and minimisers are usually outside the feasible set until late.

Local ε -statement

► there exists $\bar{\varepsilon} > 0$ such that for all $\varepsilon \in (0, \bar{\varepsilon}]$, the function F_ε has a local minimiser x_ε close to the constrained local minimiser x^* , and $x_\varepsilon \rightarrow x^*$ as $\varepsilon \downarrow 0$.

► for all $\varepsilon \in (0, \bar{\varepsilon}]$, the point x_ε is typically outside X .

sequential penalty method
is external penalty method



suitability conceptually simple, good for first constructions and initial outer steps, but not the best high-accuracy practical method.

questions Exercise 28; 2016 Q2 and Q4; 2018 Q4.

Derivation sketch

► start from the discontinuous hard penalty

$$q(x) = \begin{cases} 0, & x \in X, \\ +\infty, & x \notin X. \end{cases}$$

► then the ideal constrained cost is formally $F(x) = f(x) + q(x)$, but numerically unusable.

- ▶ replace q by a finite penalty $p(x)$ measuring equality and inequality violation, and define

$$F_\varepsilon(x) = f(x) + \frac{1}{\varepsilon}p(x).$$

- ▶ as $\varepsilon \downarrow 0$, the penalty term dominates, so minimising F_ε forces feasibility more strongly, which explains both the convergence to the constrained problem and the worsening conditioning.

2.1.2) Sequential Augmented Lagrangian Functions

for P_1 ,

$$L_a(x, \lambda) = f(x) + \lambda^\top g(x) + \frac{1}{\varepsilon} \|g(x)\|^2.$$

- ▶ if the correct multiplier were known, then minimising $L_a(x, \lambda^*)$ would already encode the equality constraint locally, so it avoids the worst ill-conditioning of pure penalties.
- ▶ $\varepsilon > 0$ is still the penalty parameter, and **smaller ε** means **stronger weight** on $\|g(x)\|^2$.

complexity $N_{\text{outer}} \times \text{cost}(\text{inner solver})$

conditions (i) equality-constrained P_1 ; (ii) local theorem at (x^*, λ^*) : regular point + second-order sufficient condition; (iii) practical algorithm updates λ_k toward λ^* ; (iv) $\varepsilon_k > 0$, usually nonincreasing.

Meaning of ε

- ▶ pure penalty method pushes $\varepsilon_k \rightarrow 0$ aggressively; augmented Lagrangian usually does **not** need that, the multiplier update already helps enforce $g(x) = 0$.
- ▶ if the constraint residual is dropping well, keep $\varepsilon_{k+1} = \varepsilon_k$; if not, reduce ε_k so the penalty term becomes stronger.

Outer updates use

$$\lambda_{k+1} = \lambda_k + \frac{2}{\varepsilon_k} g(x_k),$$

and

$$\varepsilon_{k+1} = \beta \varepsilon_k, \quad \beta = \begin{cases} 1, & \|g(x_{k+1})\| \leq \frac{1}{4} \|g(x_k)\|, \\ < 1, & \text{otherwise.} \end{cases}$$

- ▶ so ε_k is reduced **only when needed**, not automatically at every outer step.

Local ε -statement

- ▶ there exists $\bar{\varepsilon} > 0$ such that for all $\varepsilon \in (0, \bar{\varepsilon}]$, the point x^* is an unconstrained local minimiser of $L_a(x, \lambda^*)$.

► conversely, if x^* is an unconstrained local minimiser of $L_a(x, \lambda^*)$ and $g(x^*) = 0$, then x^* is a strict constrained local minimiser of P_1 .

suitability one of the most useful constrained-to-unconstrained bridges.

2.1.3) Exact Penalty Method

for P_1 ,

$$\lambda(x) = - \left[\frac{\partial g(x)}{\partial x} \left(\frac{\partial g(x)}{\partial x} \right)^\top \right]^{-1} \frac{\partial g(x)}{\partial x} \nabla f(x).$$

$$G(x) = f(x) + \lambda(x)^\top g(x) + \frac{1}{\varepsilon} \|g(x)\|^2$$

complexity one unconstrained minimisation in principle, but each evaluation can involve matrix inversion with size tied to the number of constraints.

conditions (i) equality-constrained P_1 ; (ii) $\bar{X} \subset \tilde{X}$ compact; (iii) $\frac{\partial g(x)}{\partial x} \left(\frac{\partial g(x)}{\partial x} \right)^\top$ invertible on $\tilde{X}$; (iv) global theorem: unique global minimiser in $X \cap \bar{X}$ and interior point of $\bar{X}$; (v) choose $\varepsilon \in (0, \bar{\varepsilon}]$.

Meaning of ε

- $\varepsilon > 0$ is not updated in an outer sequence here, instead, we can choose it **small enough once**.
- smaller ε means the penalty term $\frac{1}{\varepsilon} \|g(x)\|^2$ is stronger.
- there exists $\bar{\varepsilon} > 0$ such that for all $\varepsilon \in (0, \bar{\varepsilon}]$, the global theorem and the local theorem above hold for G .

Main drawbacks

- matrix inversion can be ill-conditioned, and the function may not be regular enough for fast methods like Newton.

suitability mainly theoretical or for small problems with few constraints and good regularity.

questions Exercise 29, Exercise 42.

Derivation sketch

- for equality constraints, start from the first-order condition of the Lagrangian

$$\nabla f(x) + \left(\frac{\partial g(x)}{\partial x} \right)^\top \lambda = 0, \quad g(x) = 0.$$

- multiply the first equation by $\frac{\partial g(x)}{\partial x}$:

$$\frac{\partial g(x)}{\partial x} \nabla f(x) + \frac{\partial g(x)}{\partial x} \left(\frac{\partial g(x)}{\partial x} \right)^\top \lambda = 0.$$

- at a regular point, $\frac{\partial g(x)}{\partial x} \left(\frac{\partial g(x)}{\partial x} \right)^\top$ is invertible, so

$$\lambda(x) = - \left[\frac{\partial g(x)}{\partial x} \left(\frac{\partial g(x)}{\partial x} \right)^\top \right]^{-1} \frac{\partial g(x)}{\partial x} \nabla f(x).$$

- substitute this $\lambda(x)$ into the penalised objective to obtain the exact penalty function.

2.1.4) Exact Augmented Lagrangian Functions

$$S(x, \lambda) = f(x) + \lambda^\top g(x) + \frac{1}{\varepsilon} \|g(x)\|^2 + \eta \left\| \frac{\partial g(x)}{\partial x} \nabla_x L(x, \lambda) \right\|^2.$$

- this enforces both feasibility and the **first-order condition** in an augmented variable space.

complexity unconstrained minimisation in the augmented space (x, λ) ; dimension increases to $n + m$.

conditions (i) mainly equality-constrained P_1 ; (ii) $\eta > 0$ fixed; (iii) global theorem: $\bar{X}$ compact, x^* unique global minimiser in $X \cap \bar{X}$, x^* interior, $\lambda^* \in \Lambda$ compact; (iv) local theorem: $\bar{X} \times \Lambda \subset \tilde{X} \times \mathbb{R}^m$ compact; (v) choose $\varepsilon \in (0, \bar{\varepsilon}]$.

Exactness statement

- there exists $\bar{\varepsilon} > 0$ such that, for every $\varepsilon \in (0, \bar{\varepsilon}]$, S is an exact reformulation of P_1 .

global $\Rightarrow (x^*, \lambda^*)$ is the unique global minimiser of S on the chosen compact set.

local $\Rightarrow$ if (x^*, λ^*) is a local minimiser of S , then x^* is a constrained local minimiser of P_1 .

suitability when exactness is desired but direct inversion inside the penalty function is unattractive.

questions Exercise 39; 2019 Q3.

Derivation sketch

- ▶ the exact penalty function enforces feasibility through $g(x)$ and the multiplier term. The augmented exact version also penalises violation of the first-order condition.
- ▶ that is why the extra squared term involves

$$\frac{\partial g(x)}{\partial x} \nabla_x L(x, \lambda).$$

- ▶ at a regular feasible KKT point this term vanishes, so the local minima of the augmented function recover the constrained local minima for sufficiently small ε .

2.2) Direct Constrained Methods**2.2.1) Recursive Quadratic Programming**

$$\min_s \nabla f(x_k)^\top s + \frac{1}{2} s^\top \nabla_{xx}^2 L(x_k, \lambda_k) s \quad \text{subject to} \quad g(x_k) + \frac{\partial g(x_k)}{\partial x} s = 0.$$

- ▶ this is obtained by combining a quadratic model of the Lagrangian with a first-order linearisation of the constraints.
- ▶ the resulting KKT system has the block form

$$\begin{bmatrix} \nabla_{xx}^2 L(x_k, \lambda_k) & \left(\frac{\partial g(x_k)}{\partial x} \right)^\top \\ \frac{\partial g(x_k)}{\partial x} & 0 \end{bmatrix} \begin{bmatrix} s_k \\ \mu_k \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) \\ g(x_k) \end{bmatrix}.$$

complexity one quadratic-programming solve per outer iteration; dense medium-scale cost is high, commonly $O(n^3)$ -class.

conditions (i) start near a strict constrained local minimiser x^* ; (ii) regular point; (iii) second-order sufficient condition; (iv) KKT system nonsingular; (v) $f, g \in C^3$ only if the quadratic-rate conclusion is needed.

d_k the step s_k produced by the quadratic-programming subproblem.

α_k in the local theory, the full step is natural; practical globalised versions may add a line search or merit function.

quadratic convergence if f and g are three times differentiable, the local rate is **quadratic**.

general convergence local rate is **superlinear** under the theorem hypotheses; practical globalisation uses line search / merit function.

suitability medium-scale constrained optimisation where fast local convergence matters and second-derivative information is available or can be approximated.

Derivation sketch

► set $\delta = x - x_k$. Expand the Lagrangian to second order around x_k and linearise the constraints:

$$L(x, \lambda_k) \approx L(x_k, \lambda_k) + \nabla f(x_k)^\top \delta + \frac{1}{2} \delta^\top \nabla_{xx}^2 L(x_k, \lambda_k) \delta,$$

$$g(x) \approx g(x_k) + \frac{\partial g(x_k)}{\partial x} \delta.$$

► after dropping the constant terms, the local model becomes the quadratic-programming subproblem used to compute $\delta = s_k$.

► writing the first-order conditions of that QP produces the block KKT system shown above.

2.2.2) Interior-point method (Extension)

► for inequality-constrained problems, stay **strictly inside** the feasible set and approach the boundary gradually.

$$\min_x f(x) - \mu \sum_{i=1}^p \log(-h_i(x)), \quad h_i(x) < 0, \quad \mu > 0.$$

► when μ is large $\rightarrow$ barrier is strong $\rightarrow$ iterate stays far from the boundary.

► when $\mu \downarrow 0 \rightarrow$ barrier weakens $\rightarrow$ the solution moves toward the true constrained optimum.

► with slack variables $h(x) + s = 0, s > 0$, the perturbed complementarity condition is

$$s\lambda \mathbf{1} = \mu \mathbf{1}, \quad s > 0, \lambda > 0.$$

► each outer stage fixes μ , then a Newton step is applied to the perturbed KKT system.

► best known for large sparse constrained problems, especially when active-set switching would be expensive.

quick comparison with SQP

► SQP solves a sequence of QP subproblems; interior-point solves a sequence of barrier / perturbed-KKT systems.

Appendix A) Question Map

Compact set / level set / radially unbounded Exercises: Exercise 1, Exercise 6, Exercise 8, Exercise 17.

Past papers: 2016 Q1.

Stationary points + Hessian classification Exercises: Exercise 2, Exercise 3, Exercise 6, Exercise 16.

Past papers: 2015 Q1, 2022 Q1.

Exact line search Exercises: Exercise 1, Exercise 3, Exercise 4, Exercise 16, Exercise 22, Exercise 23.

Past papers: 2015 Q2, 2016 Q1, 2018 Q1, 2023 Q4.

Gradient method Exercises: Exercise 1, Exercise 4, Exercise 11, Exercise 12, Exercise 22, Exercise 23.

Past papers: 2015 Q2, 2016 Q1, 2017 Q1, 2018 Q1, 2023 Q4.

Newton-Raphson / Newton method Exercises: Exercise 2, Exercise 7, Exercise 9, Exercise 15, Exercise 18, Exercise 26. Past papers: 2015 Q1, 2019 Q1, 2020 Q4, 2021 Q4, 2022 Q1.

Modified Newton / Newton-gradient Exercises: Exercise 18, Exercise 26. Past papers: 2015 Q1, 2022 Q1.

Trust region Exercises: none explicit. Past papers: none explicit.

Q-conjugacy / conjugate directions Exercises: none explicit. Past papers: 2023 Q4.

Conjugate gradient / Polak-Ribiere Exercises: none explicit. Past papers: 2023 Q4 (closest related conjugacy question).

Quasi-Newton / DFP / BFGS Exercises: none explicit. Past papers: none explicit.

Coordinate directions / direct search Exercises: Exercise 5. Past papers: none explicit.

Finite-difference approximation Exercises: none explicit. Past papers: none explicit.

Simplex method (not LP simplex) Exercises: none explicit. Past papers: none explicit.

Admissible set / active constraints / regular points Exercises: Exercise 27, Exercise 30, Exercise 37, Exercise 38. Past papers: 2022 Q3, 2023 Q2.

KKT conditions Exercises: Exercise 27, Exercise 41, Exercise 44. Past papers: 2022 Q3, 2023 Q2.

Sequential penalty Exercises: Exercise 28. Past papers: 2016 Q2, 2016 Q4, 2018 Q4.

Sequential augmented Lagrangian Exercises: none explicit. Past papers: none explicit.

Exact penalty Exercises: Exercise 29, Exercise 42. Past papers: none explicit.

Exact augmented Lagrangian Exercises: Exercise 39. Past papers: 2019 Q3.

Recursive quadratic programming Exercises: none explicit. Past papers: none explicit.

Appendix B) Pseudocode Bank

B.1) Unconstrained Methods

Exact line search

Input: current point x_k , direction d_k .

1. Build $\phi(\alpha) = f(x_k + \alpha d_k) - f(x_k)$.
2. Solve $\min_{\alpha \geq 0} \phi(\alpha)$ exactly.
3. Set α_k to the exact minimiser.
4. Update $x_{k+1} = x_k + \alpha_k d_k$.

Armijo backtracking

Input: $a > 0, \sigma \in (0, 1), \gamma \in (0, 1/2)$.

1. Set $\alpha = a$.
2. Test $f(x_k + \alpha d_k) - f(x_k) \leq \gamma \alpha \nabla f(x_k)^\top d_k$.
3. If true, accept $\alpha_k = \alpha$.
4. If false, shrink $\alpha \leftarrow \sigma \alpha$ and repeat.

Gradient method

1. Compute $g_k = \nabla f(x_k)$.
2. If $g_k = 0$, stop.
3. Set $d_k = -g_k$.
4. Choose α_k by line search.
5. Update $x_{k+1} = x_k + \alpha_k d_k$.

Pure Newton method

1. Compute $g_k = \nabla f(x_k)$ and $H_k = \nabla^2 f(x_k)$.
2. Solve $H_k s_k = -g_k$.
3. Update $x_{k+1} = x_k + s_k$.
4. Stop when g_k is zero or sufficiently small.

Newton-gradient method

1. Compute g_k and H_k .
2. If H_k is singular, use $d_k = -g_k$.

3. Else solve $H_k s = -g_k$.
4. If $|g_k^T s| < \varepsilon \|g_k\| \|s\|$, use $d_k = -g_k$.
5. Else if $g_k^T s < 0$, use $d_k = s$; if $g_k^T s > 0$, use $d_k = -s$.
6. Make a line search, typically starting from $\alpha = 1$.
7. Update $x_{k+1} = x_k + \alpha_k d_k$.

Trust region

1. Build the quadratic model $q(s)$ around x_k .
2. Solve $\min_{\|s\| \leq a_k} q(s)$ and obtain s_k .
3. Compute the quality ratio $\rho_k = \text{actual reduction} / \text{predicted reduction}$.
4. If $\rho_k < 1/4$, shrink the radius; if $\rho_k > 3/4$ and $\|s_k\| = a_k$, enlarge it; otherwise keep it.
5. Reject the step if $\rho_k \leq 0$; otherwise accept it.
6. Set $x_{k+1} = x_k + s_k$ when accepted.

Quadratic conjugate gradient

1. Set $d_0 = -\nabla f(x_0)$.
2. Compute $\alpha_k = -\frac{\nabla f(x_k)^T d_k}{d_k^T Q d_k}$.
3. Update $x_{k+1} = x_k + \alpha_k d_k$.
4. Compute $\beta_k = \frac{\nabla f(x_{k+1})^T Q d_k}{d_k^T Q d_k}$.
5. Set $d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k$.
6. Stop after at most n steps for SPD quadratic problems.

Polak-Ribiere

1. Set $g_0 = \nabla f(x_0)$, $d_0 = -g_0$.
2. Run a line search along d_k .
3. Update $x_{k+1} = x_k + \alpha_k d_k$.
4. Compute $g_{k+1} = \nabla f(x_{k+1})$.
5. Form $\beta_k = \frac{g_{k+1}^T (g_{k+1} - g_k)}{\|g_k\|^2}$.
6. Set $d_{k+1} = -g_{k+1} + \beta_k d_k$; restart if required.

BFGS

1. Start with $H_0 > 0$, usually I .
2. Compute $g_k = \nabla f(x_k)$; if zero, stop.

3. Set $d_k = -H_k g_k$.
4. Perform a sufficiently accurate line search to get α_k .
5. Update $x_{k+1} = x_k + \alpha_k d_k$.
6. Form $\delta_k = x_{k+1} - x_k$, $\gamma_k = g_{k+1} - g_k$, then update H_k .

B.2) Constrained Methods

Sequential penalty method

1. Choose $\varepsilon_1 > \varepsilon_2 > \dots \rightarrow 0$.
2. Build $F_{\varepsilon_k}(x) = f(x) + \frac{1}{\varepsilon_k} p(x)$.
3. Minimise F_{ε_k} as an unconstrained problem.
4. Use the current minimiser as the starting point for the next smaller ε_k .
5. Stop when constraint violation is sufficiently small.

Sequential augmented Lagrangian

1. Start from $x_0, \lambda_1, \varepsilon_1$.
2. Minimise $L_a(x, \lambda_k)$ with respect to x .
3. Update the multiplier estimate λ_{k+1} .
4. Keep or reduce ε_k depending on how well the constraint residual drops.
5. Repeat until both feasibility and stationarity are satisfactory.

Exact penalty

1. Build the exact penalty function $G(x)$ from the derivative matrix $\frac{\partial g(x)}{\partial x}$.
2. Check the required inverse exists locally.
3. Minimise $G(x)$ as an unconstrained problem.
4. Verify the obtained point is feasible; then it corresponds to a constrained local minimum.

Exact augmented Lagrangian

1. Build $S(x, \lambda)$ in the augmented space.
2. Minimise S jointly in x and λ .
3. Read the constrained solution from the minimiser of S .
4. Use this route when you want exactness without explicitly inverting the derivative-based blocks appearing in the exact penalty formula.

Recursive quadratic programming

1. Linearise the constraints at x_k .
2. Build the quadratic model of the Lagrangian at (x_k, λ_k) .
3. Solve the resulting QP subproblem to get s_k and updated multipliers.
4. Take the step (full locally, or globalised in practical variants).
5. Repeat until the KKT residual is small.

Appendix C) Comparison Tables

Unconstrained methods at a glance

- ▶ **gradient** $\Rightarrow$ needs ∇f ; low cost; safest basic descent; slow when the condition number is large.
- ▶ **Newton** $\Rightarrow$ needs $\nabla f, \nabla^2 f$; high dense cost; locally quadratic and one-step on SPD quadratics.
- ▶ **newton-gradient** $\Rightarrow$ needs $\nabla f, \nabla^2 f$; high dense cost; practical globalised Newton with gradient fall-back.
- ▶ **trust region** $\Rightarrow$ needs $f, \nabla f, \nabla^2 f$; high dense cost; robust Newton globalisation using a constrained quadratic model and trust-radius update, with no separate line search in the basic form.
- ▶ **conjugate gradient** $\Rightarrow$ needs ∇f plus SPD quadratic structure; medium cost; finite termination on quadratics with light storage.
- ▶ **polak-Ribière** $\Rightarrow$ needs ∇f ; low cost; nonquadratic CG flavour with restart flexibility.
- ▶ **DFP / BFGS** $\Rightarrow$ needs ∇f ; medium cost; inverse-Hessian approximation, with BFGS the practical favourite.
- ▶ **derivative-free methods** $\Rightarrow$ need only function values; cost depends on trial points; mainly for black-box small-dimensional problems.

Constrained methods at a glance

- ▶ **sequential penalty** $\Rightarrow$ force feasibility with $1/\varepsilon$ penalties; simple, but conditioning worsens as $\varepsilon \rightarrow 0$.
- ▶ **sequential augmented Lagrangian** $\Rightarrow$ mix penalties with multiplier estimates; much better conditioning than pure penalties.
- ▶ **exact penalty** $\Rightarrow$ solve one unconstrained problem in principle; exact, but can need unstable inverses and may be less smooth.
- ▶ **exact augmented Lagrangian** $\Rightarrow$ exactness in augmented variables; avoids explicit inversion, but increases dimension.
- ▶ **RQP / SQP** $\Rightarrow$ solve a QP at each step; strongest local speed, but also the heaviest modelling and per-step cost.